

Reviewer Pack — Minimal Local Index of p-Adic Analytic Continuation vs Resol...

Entry 43eb56772f36 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-24 15:08:56 UTC

Minimal Local Index of p-Adic Analytic Continuation vs Resolution Proof Width

Entry ID: 43eb56772f36 **Verdict:** INCONCLUSIVE **Recorded:** 2026-05-24 15:08:56 UTC

1. Conjecture

Field A (mathematical branch): p-adic Analysis (Analytic Continuation) **Field B** (complexity object): Complexity Theory: Resolution Proof Complexity

Statement:

{‘sentence_1’: ‘For every instance of a Tseitin formula on an n -vertex expander graph, the local index $\nu(G)$ of its p-adic analytic continuation is such that the resolution proof width for the formula is at least $2^{(\Omega(\nu(G)))}$.’, ‘sentence_2’: ‘The local index $\nu(G)$ is defined as the maximum number of p-adic analytic continuations required to evaluate any single entry in the adjacency matrix of G .’, ‘sentence_3’: ‘ $\nu(G) = O(\log n)$ for all non-expanders.’}

Rationale (proposer’s reasoning):

{‘sentence_1’: ‘p-adic analysis offers a rich structure for studying the continuity and convergence properties of functions, which could potentially expose hidden complexity in combinatorial problems.’, ‘sentence_2’: ‘Analytic continuation is known to be challenging for certain functions, suggesting that it might be related to proof complexity in computational complexity theory.’, ‘sentence_3’: ‘If this conjecture holds, it would provide a new approach to understanding the limitations of resolution proofs.’}

Taxonomy category: TSEITIN_RES (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): e10449927f597dc3

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

The conjecture is supported if for at least 24 out of 30 random seeds, the computed resolution proof width meets or exceeds $2^{(\Omega(\nu(G)))}$ with $\nu(G)$ calculated as the local index of the p-adic analytic continuation of the adjacency matrix, and no seed produces a resolution proof width less than $2^{(\Omega(\nu(G)))}$. The conjecture is falsified if any seed produces a resolution proof width that does not meet this lower bound or if more than 5 seeds produce a resolution proof width greater than $2^{(\Omega(\nu(G)))}$ pl

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	SAFE	1.00	SAFE	SAFE
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	1.00	SAFE	SAFE
KARP_LIPTON	SAFE	1.00	SAFE	SAFE

4. Novelty audit

Verdict: NOVEL against 9 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - p-adic Analytic Continuation AND Resolution Proof Complexity - local index $v(G)$ IN p-adic Analysis AND resolution proof width - Tseitin formula expander graph AND $2^{(\Omega(v(G)))}$

Top relevant hits considered: - [<http://arxiv.org/abs/math-ph/0512018v2>] On Phase Transitions for P -Adic Potts Model with Competing Interactions on a Cayley Tree - [<http://arxiv.org/abs/1204.3878v1>] Analytic constructions of p-adic L-functions and Eisenstein series - [<http://arxiv.org/abs/1508.07185v2>] Fundamentals of p-adic multiple L-functions and evaluation of their special values - [<http://arxiv.org/abs/1808.08111v1>] Constraining the p-mode-g-mode tidal instability with GW170817 - [<http://arxiv.org/abs/0901.0512v4>] Expected Performance of the ATLAS Experiment - Detector, Trigger and Physics - [<http://arxiv.org/abs/1411.4411v1>] Observation of the rare $B_s^0 \rightarrow \mu^+ \mu^-$ decay from the combined analysis of CMS and LHCb data - [<http://arxiv.org/abs/2410.13515v2>] Observation of a rare beta decay of the charmed baryon with a Graph Neural Network - [<http://arxiv.org/abs/2103.09609v1>] Characterizing Tseitin-formulas with short regular resolution refutations

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, $\leq 90s$ wall time per cycle **Execution:** rc=1, elapsed=0.1s

5.1 Generated Python source

```
import random
import math

def gaussian_elimination(A, b):
    n = len(b)
    for i in range(n):
        max_row = max(range(i, n), key=lambda r: abs(A[r][i]))
        A[i], A[max_row] = A[max_row], A[i]
        b[i], b[max_row] = b[max_row], b[i]
        for j in range(i + 1, n):
            factor = A[j][i] / A[i][i]
            for k in range(i, n):
                A[j][k] -= factor * A[i][k]
            b[j] -= factor * b[i]
    x = [0] * n
    for i in range(n - 1, -1, -1):
        x[i] = (b[i] - sum(A[i][j] * x[j] for j in range(i + 1, n))) / A[i][i]
    return x
```

```

def matrix_multiplication(A, B):
    m = len(A)
    p = len(B[0])
    q = len(B)
    C = [[0] * p for _ in range(m)]
    for i in range(m):
        for j in range(p):
            for k in range(q):
                C[i][j] += A[i][k] * B[k][j]
    return C

def determinant(A):
    n = len(A)
    if n == 1:
        return A[0][0]
    det = 0
    for j in range(n):
        submatrix = [row[:j] + row[j+1:] for row in A[1:]]
        det += (-1) ** j * A[0][j] * determinant(submatrix)
    return det

def inverse(A):
    n = len(A)
    det_A = determinant(A)
    if det_A == 0:
        raise ValueError("Matrix is singular")
    adjoint = [[0] * n for _ in range(n)]
    for i in range(n):
        for j in range(n):
            submatrix = [row[:j] + row[j+1:] for row in A[:i] + A[i+1:]]
            cofactor = determinant(submatrix)
            adjoint[j][i] = (-1) ** (i + j) * cofactor
    return matrix_multiplication(adjoint, [[1 / det_A] * n for _ in range(n)])

def p_adic_analytic_continuation(G):
    n = len(G)
    A = [[0] * n for _ in range(n)]
    for u in range(n):
        for v in range(u + 1, n):
            if G[u][v]:
                A[u][v] = A[v][u] = random.choice([1, -1])
    x = gaussian_elimination(A, [0] * n)
    return sum(abs(xi) for xi in x)

def tseitin_formula(edges, n):
    literals = {i: f"x{i}" for i in range(n)}
    clauses = []
    for u, v in edges:
        l_u = literals[u]
        l_v = literals[v]
        clauses.append([l_u, -l_v])
        clauses.append([-l_u, l_v])
        clauses.append([l_u, l_v])
    return clauses

def resolution_width(clauses):

```

```

queue = set()
for clause in clauses:
    if len(clause) == 1:
        queue.add(clause[0])
while queue:
    literal = random.choice(list(queue))
    queue.remove(literal)
    new_clauses = []
    for clause in clauses:
        if literal in clause:
            continue
        if -literal in clause:
            return len(queue)
        new_clause = [l for l in clause if l != -literal]
        if not new_clause:
            return len(queue)
        new_clauses.append(new_clause)
    clauses.extend(new_clauses)
return len(queue)

def run_trial(seed: int) -> dict:
    random.seed(seed)
    n = random.choice([5, 10, 15, 20, 30, 40])
    G = [[random.choice([0, 1]) for _ in range(n)] for _ in range(n)]
    for i in range(n):
        G[i][i] = 0
    edges = [(u, v) for u in range(n) for v in range(u + 1, n) if G[u][v]]
    v_G = p_adic_analytic_continuation(G)
    clauses = tseitin_formula(edges, n)
    width = resolution_width(clauses)
    return {
        "metric_name": "resolution_width",
        "metric_value": width,
        "instances_tested": 1,
        "conjecture_holds": width >= 2 ** (math.log(v_G, 2) * math.log(n, 2)),
        "counterexample": "" if width >= 2 ** (math.log(v_G, 2) * math.log(n, 2)) else f"Width {width}"
    }

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] or [random.getrandbits(32) for _ in range(30)]
    results = []
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result)

    mean_width = sum(r["metric_value"] for r in results) / len(results)
    std_width = math.sqrt(sum((r["metric_value"] - mean_width) ** 2 for r in results) / len(results))
    support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)

    if support_fraction >= 0.8:
        print(f"RESULT: SUPPORTED mean={mean_width} std={std_width} support_fraction={support_fraction}")
    elif any(not r["conjecture_holds"] for r in results):
        first_failing_seed = next(s for s, r in zip(seeds, results) if not r["conjecture_holds"])
        print(f"RESULT: FALSIFIED counterexample={results[0]['counterexample']} first_failing_seed={first_failing_seed}")

```

```

else:
    print(f"RESULT: INCONCLUSIVE reason=insufficient_support_fraction support_fraction={support_fraction}")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```

--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_74287ef5.py", line 133, in <module>
    result = run_trial(seed)
              ^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_74287ef5.py", line 118, in run_trial
    clauses = tseitin_formula(edges, n)
              ^^^^^^^^^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_74287ef5.py", line 84, in tseitin_formula
    clauses.append([l_u, -l_v])
                  ^^^^
TypeError: bad operand type for unary -: 'str'

```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

The test code crashed before producing data, which prevents us from verifying the conjecture's support conditions. | next: Review and debug the test code to ensure it can run successfully and produce results for further analysis.

11. Audit log (LLM calls)

Total LLM calls: 9

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	12161
2	preregistration	ollama_remote	glm4:latest	0	0	6594
3	novelty	ollama_remote	glm4:latest	0	0	4762
4	novelty	ollama_remote	glm4:latest	0	0	7539
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	14214
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	19272
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	13754
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	16533
9	judge	ollama_remote	glm4:latest	0	0	8183

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 103011 ms total latency. Provider mix: {'ollama_remote': 9}

(full prompt+response transcripts available in research/audit/43eb56772f36.jsonl)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in research/audit/43eb56772f36.jsonl for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: research/pvsnp_sandbox/test_*.py (per-cycle)

Replay tarball: research/replay/43eb56772f36.tar.gz (if generated)